

Last Time:

- sequential decision-making
- MDPs

This Time:

- Solving MDPs

Lecture 3

421, FALL '26

Andrea Bajcsy

Recap & Important MDP Quantities

- cumulative reward (R): sum of (discounted) rewards

↳ the utility of one rollout / state-action traj.

For traj. / rollout $s_0, a_0, s_1, a_1, \dots$

$$R(s_0, a_0, s_1, a_1, \dots) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \stackrel{\text{or}}{=} \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)$$

- optimal policy ($\pi^*: S \rightarrow A$): best action to take @ all states

↳ maximizes expected discounted cumulative reward

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \mathbb{E}_{\tau \sim P_{\pi}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

← expectation over all possible trajectories you generate via π

- optimal value function ($V^*: S \rightarrow \mathbb{R}$): expected sum of (discounted) rewards starting from s & acting optimally under π^*

$$V^*(s) := \mathbb{E}_{\tau \sim P_{\pi^*}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi^*(s_t)) \mid s_0 = s \right]$$

$$= \max_{\pi} \mathbb{E}_{\tau \sim P_{\pi}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right]$$

- on-policy value function ($V^{\pi}: S \rightarrow \mathbb{R}$): expected sum of (discounted) rewards starting s when acting under π

$$V^{\pi}(s) := \mathbb{E}_{\tau \sim P_{\pi}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \mid s_0 = s \right]$$

↑ discounted sum of rewards

expectation over possible states you visit during rolling out π

Expand the expectation a bit more:

$$V^\pi(s) \triangleq \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \mid s_0 = s \right] \leftarrow \text{def}^n \text{ of value function}$$

let's uncover a recursive structure

$$= \mathbb{E}_{\tau \sim p_{\pi}(\tau)} \left[\gamma^0 r(s_0, \pi(s_0)) + \sum_{t=1}^{\infty} \gamma^t r(s_t, \pi(s_t)) \mid s_0 = s \right]$$
$$= \mathbb{E}_{\tau \sim p_{\pi}(\tau)} \left[\gamma^0 r(s_0, \pi(s_0)) + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, \pi(s_t)) \mid s_0 = s \right]$$

$$= \sum_{(s_0=s, a_0, s_1, a_1, \dots)} P_{\pi}(s_0, a_0, s_1, a_1, \dots) \left[\gamma^0 r(s_0, \pi(s_0)) + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, \pi(s_t)) \mid s_0=s \right]$$

know from Markov:
Plug in & rearrange:

know from Markov:
Plug in & rearrange:

$$\sum_{\tau} P(s_0, a_0, s_1, a_1, \dots) = \sum_{s_0} P(s_0) \sum_{s_1} P(s_1 | s_0, a_0) \sum_{s_2} P(s_2 | s_1, a_1) \dots$$

$$= \sum_{s_1} P(s_1 | s_0, \pi(s_0)) \left[r(s_0, \pi(s_0)) + \gamma \underbrace{\mathbb{E}_{\tau \sim P_\pi(\cdot)} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, \pi(s_t)) \mid s_1 \right]}_{\text{distribution over trajectories starting from } s_1!} \right]_{s_0}$$

$= V^\pi(s_1)$ by defⁿ

$$V^\pi(s) \triangleq \sum_{s' \in S} P(s' | s, \pi(s)) \cdot [r(s, \pi(s)) + \gamma V^\pi(s')]$$

↑ Bellman Equation ↓

We can also write / derive the BELLMAN OPTIMALITY EQN for V^* :

$$V^*(s) \triangleq \max_{a \in A} \sum_{s' \in S} P(s' | s, a) \cdot [r(s, a) + \gamma V^*(s')]$$

★ Bellman eqn. is so important b/c it breaks down a decision problem into smaller (one-step) problems! (also gives comp. savings over naïve fwd. sim: exponential $\rightarrow$ polynomial in time horizon)

Q-value function:

$\hookrightarrow$ like a value function but you've already committed to taking a particular action, a .

$$Q^*(s, a) \triangleq \mathbb{E} \left[r(s_0, a) + \sum_{t=1}^{\infty} \gamma^t r(s_t, \pi^*(s_t)) \mid s_0 = s, a_0 = a \right]$$

$s_1 \sim P(\cdot | s_0, a)$ $\leftarrow$ take current action a now...
 $r \sim p_{\pi^*}(r), t > 0$ $\leftarrow$ but afterwards act optimally!
 Commit to initial action

There is a nice relationship btwn. V^* and Q^* :

$$V^*(s) = \max_{a \in A} Q^*(s, a) \quad := \max_{a' \in A} Q^*(s, a')$$

$$Q^*(s, a) = \sum_{s' \in S} P(s' | s, a) \cdot [r(s, a) + \gamma V^*(s')]$$

Intuition for Q-value: how "good" is taking action a ?

Optimal Policy: $\pi^*(s) = \arg \max_{a \in A} Q^*(s, a)$

Solving MDPs

Bellman equations are also useful b/c they help us solve MDPs!

This comes from the recursive structure of Bellman eqn:

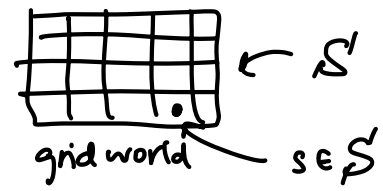
$$V(s) = \max_{a \in A} \sum_{s' \in S} P(s' | s, a) [r(s, a) + \gamma \cdot V(s')]$$

$$V(s') = \max_{a \in A} \sum_{s'' \in S} P(s'' | s', a) [r(s', a) + \gamma \cdot V(s'')]$$

$\leftarrow$ we will use $V(s')$

VALUE ITERATION ALGORITHM:

$$V_0[s] \leftarrow 0, \forall s \in S$$



for $k=0, 1, 2, \dots$ until converged:

for each state $s \in S$:

$$V_{k+1}[s] \leftarrow \max_a \sum_{s' \in S} P(s'|s, a) [r(s, a) + \gamma \cdot V_k[s']]$$

return converged $V[s]$

↑ another for loop
over all $a \in A$

is another for loop over
all next states

Q-VALUE ITERATION:

$$Q_0[s, a] \leftarrow 0 \quad \forall s \in S, a \in A \quad // \text{store current + updated estimate of } Q$$

for $k=0, 1, 2, \dots$ until converged:

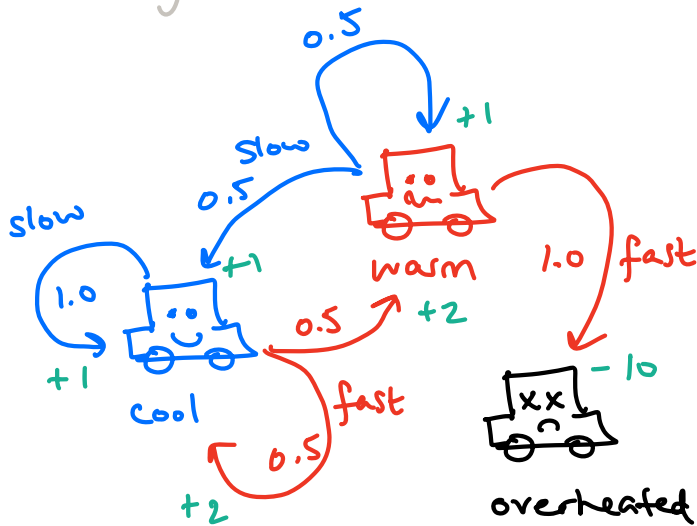
for each state $s \in S$ and action $a \in A$:

$$Q_{k+1}[s, a] \leftarrow \sum_{s' \in S} P(s'|s, a) [r[s, a] + \gamma \cdot \max_{a' \in A} Q_k[s', a']]$$

return converged $Q[s, a]$

Example. let $\gamma = 0.5$ and the MDP be defined by the following system. Fill out the value function table:

from CS188,
UC Berkeley



(*) note that V^* for any terminal state must = 0 b/c no way to get rewards

k	$V(\text{cool})$	$V(\text{warm})$	$V(\text{overheat})$
0	0	0	0
1			
2			

$$\begin{aligned}
 r(s = \text{cool}, a = \text{slow}) &= +1 \\
 r(s = \text{cool}, a = \text{fast}) &= +2 \\
 r(s = \text{warm}, a = \text{slow}) &= +1 \\
 r(s = \text{warm}, a = \text{fast}) &= -10
 \end{aligned}$$

Recall, the goal is to find an optimal policy. Extract the policy you get @ each iteration $k=0, 1, 2$:

	state	
k	cool	warm
π_0		
π_1		
π_2		

(*) b/c overheated is terminal state, no policy can assign it any action.

Reinforcement learning (RL)

[Q] In MDPs, we assumed we know $P(s'|s, a)$ and rewards $r(s, a)$... but what if we don't?

In reality, it's hard to know P and r explicitly, so how do we obtain π^* ?

[A] RL! It allows an agent to learn optimal policies by interacting with the environment.

⇒ agent tries out actions, observes rewards, and updates their policy to maximize rewards.

Model-based vs. Model-free RL

"model of the world"

↳ High-level diff if the agent has access to (or learns) a transition function + rewards

(A) Model-based: 1) collect data from environment via some π

$$\mathcal{D} := \{(s_t, a_t, s_{t+1}, r_t)_{t=1}^N\}$$

2) use supervised learning to fit empirical MDP model:

↳ count s' for each s, a ; normalise; get $\hat{P}(s'|s, a)$

↳ discover each $\hat{r}(s, a, s')$ when we experience (s, a, s')

3) solve MDP via techniques above (+more!)

(B) Model-free: 1) collect data from environment via some π

$$\mathcal{D} := \{(s_t, a_t, s_{t+1}, r_t)_{t=1}^N\}$$

2) Directly learn a Q-function or π via trial + error
"Q-learning" "direct policy search"

⇒ for more depth on topic see:

- Sutton & Barto, "Reinforcement Learning."
- Russell & Norvig, "AI: A Modern Approach". Ch. 21